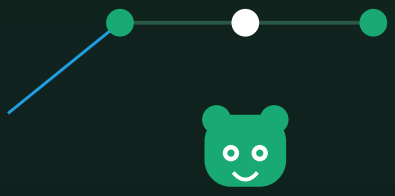
VISUAL STUDY NOTES





GitHub Notes

Every GitHub feature, the CLI, and the Desktop app, start to finish.

Beginner-friendly. Straight to the point. A full website + command reference.

CREATED BY

Sharad Verma @sharadverma638

Issues | Pull Requests | Actions | Forking | Codespaces | CLI

21 sections

30+ illustrations

120+ commands

</> **Want to learn DevOps?** ↗

github.com/sharadverma638/sv-opslibrary

Edition 2026 | Companion guide: Git Notes | by Sharad Verma

Contents

21 sections, start to finish. Follow in order, or jump to what you need.

01	**What Is GitHub?** GitHub vs. Git, why teams use it, the big picture	2
02	**Your Account & Profile** Sign-up, profile, SSH/HTTPS auth, avatar & README	4
03	**Creating a Repository** New repo, README, .gitignore, license, visibility	6
04	**Touring a Repository Page** Code tab, branches, tags, Insights, About panel	8
05	**Issues** Tracking bugs & tasks: labels, assignees, milestones, templates	10
06	**Pull Requests** Proposing changes, reviews, merge/squash/rebase methods	12
07	**Forking & Contributing** Fork › branch › PR workflow for open source	14
08	**Discussions** Q&A and conversation space outside Issues	16
09	**GitHub Actions & CI/CD** Workflows, YAML basics, triggers, runners	17
10	**Projects** Kanban-style boards and planning views	19
11	**Wiki** Long-form project documentation	21
12	**Releases & Tags** Shipping versions, changelogs, release assets	22
13	**GitHub Pages** Free static website hosting from a repo	24
14	**Packages** Publishing & installing packages via GitHub	26
15	**Codespaces** A full dev environment in your browser	28
16	**Copilot Overview** AI pair-programmer: chat, completions, PR summaries	29
17	**Security Essentials** Branch protection, Dependabot, secret scanning	31
18	**GitHub CLI (gh)** The full command-line reference	33
19	**GitHub Desktop Walkthrough** Clone, commit, push/pull & PRs without the terminal	37
20	**Best Practices** Habits that keep repos & teams healthy	39
21	**Cheat Sheet & Glossary** One-page recall + every term defined	41

How to read this guide

- **Diagrams:** Custom-drawn illustrations of every screen & flow — no screenshots.
- **Code blocks:** Dark panels show exact commands you can type, line by line.
- **Callouts:** Colored tips/notes/cautions flag beginner gotchas as they come up.

01 What Is GitHub?

The big picture before you click anything

Git is a tool that runs on **your computer** and tracks changes to your files. GitHub is a **website** that hosts copies of your Git repositories online, so you and other people can see them, back them up, and collaborate — like the difference between a word processor (Git) and Google Drive (GitHub): one does the work, the other stores and shares it.

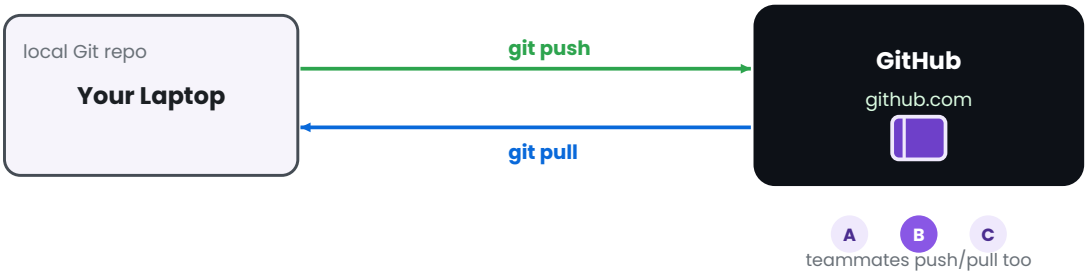


Fig 1. Your laptop keeps the full history locally; GitHub keeps the same history online, synced with git push / git pull.

What GitHub adds on top of Git

- **A home page for every project** — code, docs, and history, browsable without installing anything.
- **Issues & Pull Requests** — structured ways to discuss bugs, ideas, and proposed changes.
- **Automation (Actions)** — run tests, builds, and deployments automatically when code changes.
- **Social features** — stars, followers, forks, and public profiles that double as a portfolio.
- **Free hosting extras** — static websites (Pages), package registries, Wikis, and more.

A quick mental model

Term	Think of it as…
`Git`	A tool installed on your machine; works offline; tracks versions of files.
`GitHub`	A website built around Git; adds collaboration, hosting, and automation.
`Repository ("repo")`	A project's folder plus its entire saved history.
`Remote`	A repo's address on a server (GitHub) that your local repo talks to.

NOTE

You do **not** need to be an expert in Git to use GitHub — this guide explains every website feature, the `gh` command line, and GitHub Desktop from zero.

02 Your Account & Profile

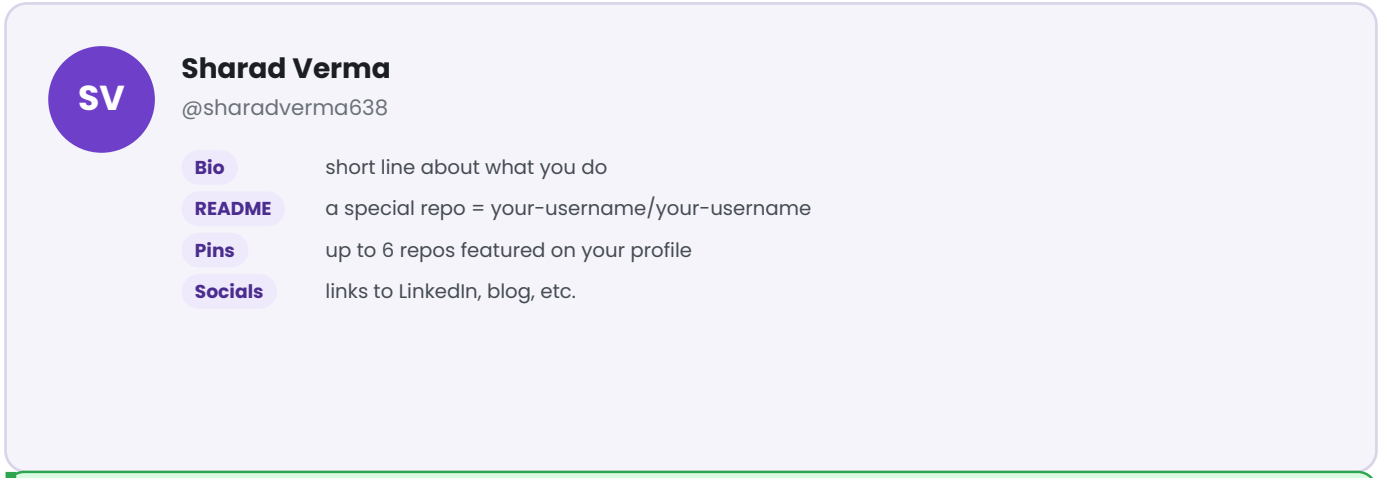
Sign up once, set up auth, you're in

Everything on GitHub starts with a free personal account. Your **username** becomes part of every URL you own, e.g. `github.com/yourname` — choose something short and professional; you can change it later, but old links can break.

Signing up

1. **Go to github.com and click Sign up**
   Enter an email, create a password, and pick a username.
2. **Verify your email**
   GitHub sends a code or link — confirm it to activate the account.
3. **Choose the Free plan**
   Free covers unlimited public/private repos, Actions minutes, and Pages — plenty to start.
4. **Land on your dashboard**
   A feed of activity from repos you watch, plus quick links to create or explore.

Setting up your profile



TIP

Create a repo named **exactly** your username (e.g. `sharadverma638/sharadverma638`) with a `README.md` — GitHub automatically renders it at the top of your profile page.

HTTPS vs. SSH — how you authenticate

When Git on your machine talks to GitHub (push/pull), it has to prove it's really you. There are two common ways to set that up:

Method	How it works
`HTTPS + token`	A Personal Access Token (PAT) stands in as your password. Scoped, revocable. Generate one under Settings › Developer settings › Personal access tokens.
`SSH key`	Generate a key pair once (`ssh-keygen`), add the public key under Settings › SSH and GPG keys. No password prompts afterward.

terminal — one-time SSH setup

```
$ ssh-keygen -t ed25519 -C "you@email.com"
$ cat ~/.ssh/id_ed25519.pub   # copy this into GitHub › SSH keys
$ ssh -T git@github.com          # test the connection
```

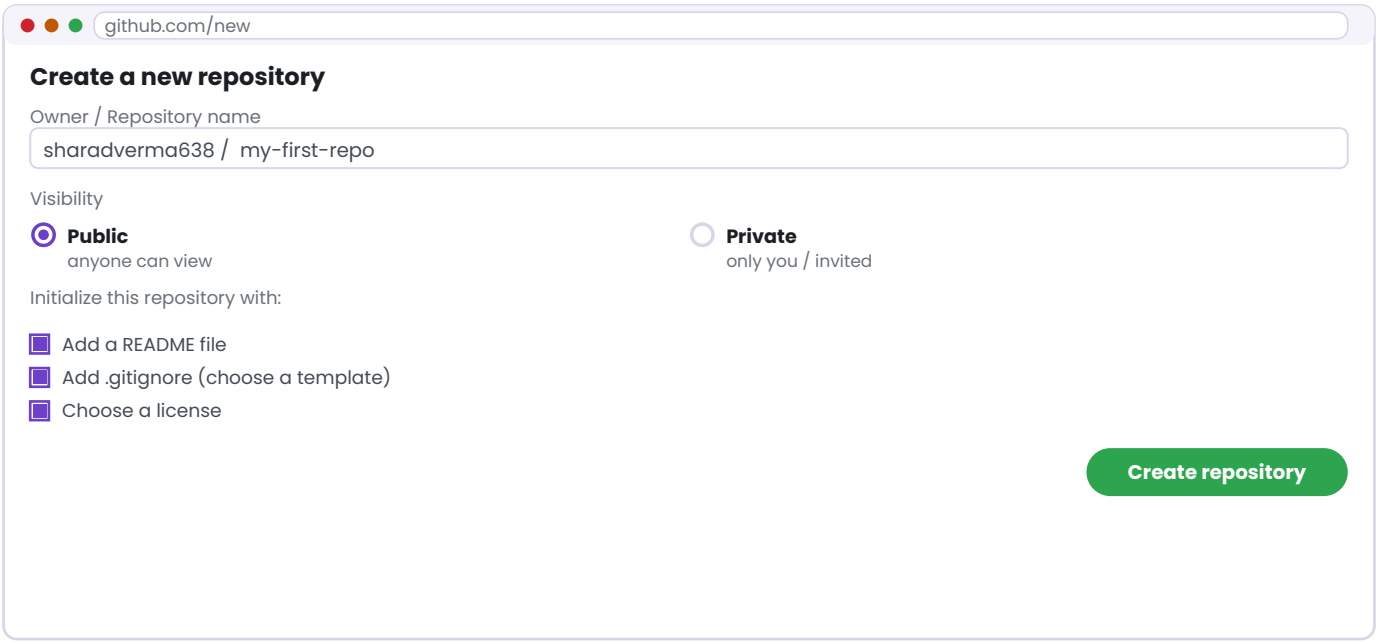
CAUTION GitHub stopped accepting plain account passwords for Git operations over HTTPS — you must use a token (or SSH key) instead. Save the token somewhere safe; GitHub shows it only once.

03 Creating a Repository

Your project's new home on GitHub

A repository (“repo”) is a project folder plus its full history. You can start one two ways: create it **on GitHub first** then clone it down, or create it **locally first** and push it up. Beginners usually find starting on GitHub easier.

The “New repository” form



Option	What it's for
`README.md`	The first thing visitors see. Explain what the project does and how to run it.
`.gitignore`	A list of files/folders Git should never track (build output, secrets, node_modules).
`LICENSE`	Legal terms for reuse. MIT and Apache-2.0 are common permissive choices.
`Public vs Private`	Public = anyone can see the code. Private = only you and people you invite.

Getting the repo onto your computer

terminal

```
$ git clone https://github.com/sharadverma638/my-first-repo.git
$ cd my-first-repo
$ git status                  # confirm you're connected
```

Already have a local project? Push it up instead:

terminal — local folder › new GitHub repo

```
$ git init
$ git add .
$ git commit -m "Initial commit"
$ git branch -M main
$ git remote add origin https://github.com/you/repo.git
$ git push -u origin main
```

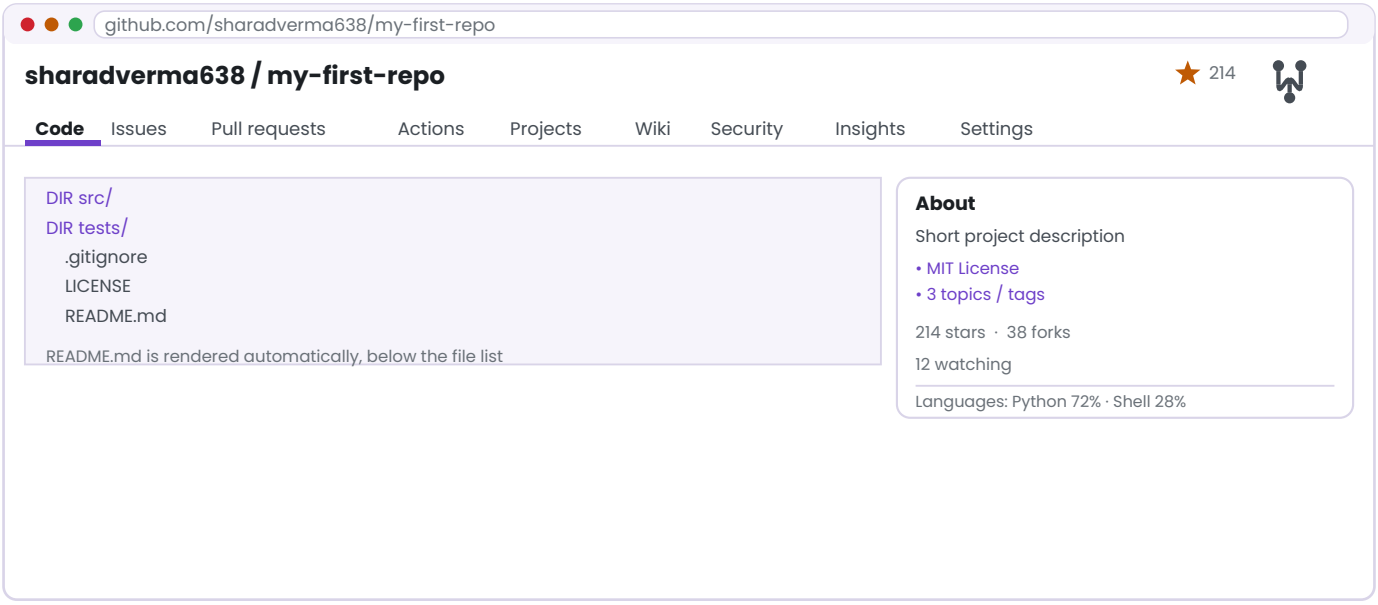
CAUTION

Create the GitHub repo **without** a README/.gitignore when you plan to push an existing local project — otherwise the histories won't line up and the first push can be rejected.

04 Touring a Repository Page

Every tab, decoded

Every repo shares the same layout. Once you recognize these tabs, you can orient yourself in **any** GitHub project in seconds, even ones you've never seen before.



Tab	What lives there
`Code`	File browser, README render, branch/tag switcher, “Add file”, clone/download button.
`Issues`	Bug reports and feature requests — covered in full in Section 5.
`Pull requests`	Proposed changes waiting for review — covered in Section 6.
`Actions`	Automated workflows (tests, builds, deploys) — covered in Section 9.
`Projects`	Kanban boards linking Issues/PRs to a plan — Section 10.
`Wiki`	Long-form docs pages separate from the code — Section 11.
`Security`	Alerts from Dependabot, code scanning, secret scanning — Section 17.
`Insights`	Contributors, commit activity graphs, traffic, dependency graph.
`Settings`	Rename/transfer/delete repo, branch protection, collaborators, webhooks.

The branch switcher & clone button

- The **branch dropdown** (top-left of the file list) switches which version of the code you're browsing — default is usually `main`.
- The green **Code** button gives you the clone URL (HTTPS/SSH), a “Download ZIP” option, and one-click “Open with GitHub Desktop” / “Create codespace.”

TIP Press `.` (a single period) while viewing any repo on GitHub.com to open it instantly in a full web-based VS Code editor — no install, no Codespace needed.

05

Issues

Tracking bugs, tasks, and ideas

An **Issue** is a single trackable item on GitHub — a bug report, a feature request, a question, or a to-do. Think of it as a sticky note pinned to the repo that people can discuss, label, assign, and eventually close.



Fig. Anatomy of an Issue: title, status pill, comment thread, and a sidebar for metadata.

Creating and organizing issues

1. **Open a new issue**

   From the Issues tab, click New issue. Give it a clear, searchable title.

2. **Describe it well**

   What happened, what you expected, and steps to reproduce (for bugs).

3. **Add metadata**

   Assignees (who's on it), Labels (bug/feature/docs), Milestone (which release), Projects (which board).

4. **Discuss & close**

   Comment to clarify; close manually or automatically when a linked PR merges.

Labels you'll see everywhere

Label	Meaning
`bug`	Something isn't working as intended.
`enhancement`	A new feature or improvement request.
`documentation`	Docs need adding, fixing, or clarifying.
`good first issue`	Scoped small enough for a first-time contributor.
`help wanted`	Maintainers would welcome outside help on this.
`wontfix`	Acknowledged, but won't be addressed.

Closing issues automatically from a commit or PR

keywords GitHub understands: close, closes, closed, fix, fixes, fixed, resolve(s/d)

```
git commit -m "Fix mobile nav overlap, closes #42"
# or in a Pull Request description:
Fixes #42
```

Issue templates

Maintainers can add template files under `.github/ISSUE_TEMPLATE/` (Markdown or YAML forms) so every bug report or feature request starts with the right questions already filled in — no more empty issues with just a title.

TIP Use GitHub's search filters right in the Issues tab: `is:issue is:open label:bug assignee:@me` finds your own open bugs instantly.

06 Pull Requests

Proposing, reviewing, and merging changes

A **Pull Request (PR)** asks the repo's maintainers to pull your changes from one branch into another — usually your feature branch into `main`. It's where code gets reviewed, discussed, tested by Actions, and finally merged.

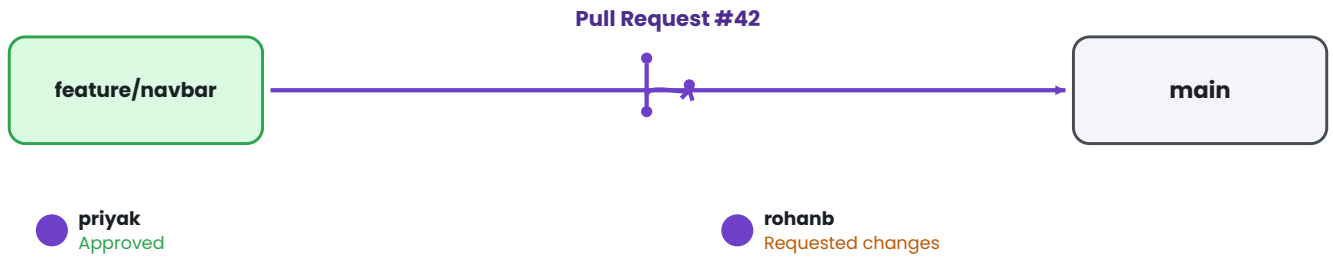


Fig. A PR proposes merging one branch into another; reviewers approve or request changes.

Opening a pull request

1. **Push your branch**
   git push -u origin feature/navbar — GitHub detects the new branch.
2. **Click “Compare & pull request”**
   A banner appears on the repo page, or use the Pull requests tab › New.
3. **Pick base ‹ compare branches**
   base = where changes land (main); compare = your branch.
4. **Write a clear description**
   What changed and why. Link related issues with “Fixes #42”.
5. **Request reviewers & open it**
   Add teammates as reviewers; GitHub notifies them.

The review tabs

Tab	What it's for
`Conversation`	The PR description, comments, review summaries, and status checks.
`Commits`	Every individual commit included in this PR, in order.
`Checks`	CI results from GitHub Actions — tests, linting, builds.
`Files changed`	A line-by-line diff. Leave inline comments on exact lines here.

The three merge methods

1 — Merge commit

Keeps every commit + adds a merge commit. Full history, branch shape visible.

2 — Squash and merge

All commits become ONE clean commit on main. Most popular default.

3 — Rebase and merge

Replays each commit onto main individually — no merge commit, linear history.

KEY IDEA Not sure which to pick? **Squash and merge** is the safest default for most teams — it keeps main's history short and readable, one commit per feature.

Draft PRs & review etiquette

- Open a **Draft pull request** to share work-in-progress without triggering a full review request.
- Reviewers leave comments inline, then submit a review as **Comment**, **Approve**, or **Request changes**.
- Branch protection rules can require 1+ approvals and passing checks before the Merge button unlocks (Section 17).

terminal

```
$ git switch feature/navbar
$ git push -u origin feature/navbar
# then open the PR on github.com, or with the CLI:
$ gh pr create --fill
```

07 Forking & Contributing

The open-source contribution workflow

A **fork** is your own personal copy of someone else's repository, on your account. Fork when you don't have write access to the original (most open-source projects) — you get full freedom to experiment, then propose your changes back via a Pull Request.

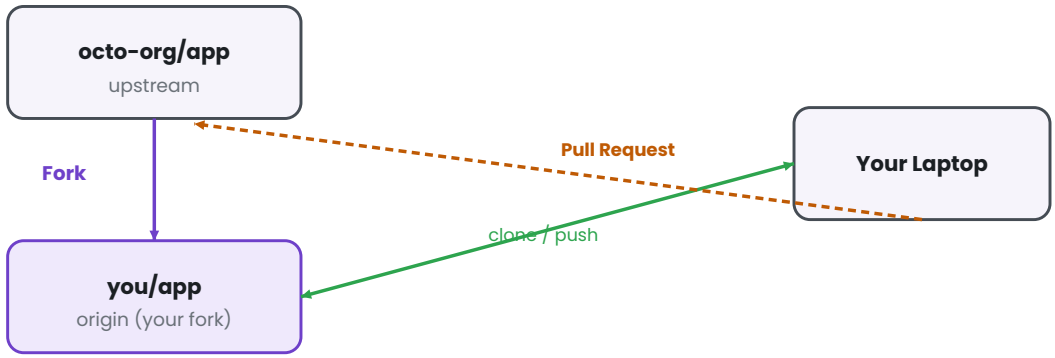


Fig. Fork the project, clone your fork, push a branch, open a PR back to upstream.

The full workflow

1. **Fork the repo**
   Click Fork (top-right of the original repo). Creates you/app on your account.
2. **Clone your fork**
   git clone https://github.com/you/app.git — not the original.
3. **Add the original as "upstream"**
   git remote add upstream https://github.com/octo-org/app.git
4. **Branch, commit, push**
   Always work on a feature branch, never directly on your fork's main.
5. **Open a PR to upstream**
   base: octo-org/app main ‹ compare: you/app feature-branch

terminal — fork workflow

```
$ git clone https://github.com/you/app.git
$ cd app
$ git remote add upstream https://github.com/octo-org/app.git
$ git switch -c fix/typo
$ git add . && git commit -m "Fix typo in README"
$ git push -u origin fix/typo
$ gh pr create --repo octo-org/app --fill
```

Keeping your fork in sync

Forks don't update automatically. Before starting new work, sync your fork's main with upstream so you're never building on stale code:

terminal

```
$ git fetch upstream
$ git switch main
$ git merge upstream/main
$ git push origin main
# or, one click on github.com: "Sync fork"
```

TIP Read the repo's `CONTRIBUTING.md` before opening a PR — most open-source projects document exact expectations for commit style, tests, and PR size.

08 Discussions

A conversation space outside Issues

Discussions are for open-ended conversation — questions, ideas, announcements, polls — things that aren't a concrete bug or task. Keeping them separate from Issues keeps your issue tracker focused on actionable work.

Category	Typical use
`Q&A`	Community questions; answers can be marked as the accepted answer.
`Ideas`	Feature suggestions and brainstorming before they become formal Issues.
`Announcements`	Maintainer-only posts — releases, roadmap updates (comments open, new posts restricted).
`General / Polls`	Open chat, or structured polls to gauge community preference.



Fig. When to use which — Issues, Discussions, and Pull Requests serve different jobs.

Enabling Discussions

- Off by default on new repos. Turn it on under **Settings › General › Features › Discussions**.
- A maintainer can convert a sprawling Issue into a Discussion (and back) when its nature changes.
- Discussions support Markdown, reactions, nested replies, and marking a reply as the answer.

NOTE Many popular open-source projects route all "how do I…" and "is this a bug or am I misusing it" questions to Discussions, reserving Issues strictly for confirmed, actionable bugs.

09

GitHub Actions & CI/CD

Automate tests, builds, and deploys

GitHub Actions runs scripts automatically when something happens in your repo — a push, a PR, a schedule. Most commonly it's used for **CI/CD**: running tests on every PR, then deploying automatically when main updates.



.github/workflows/ci.yml defines what runs, on what trigger, on what runner

Fig. Every workflow run starts from an event and ends in a pass/fail check.

Anatomy of a workflow file

.github/workflows/ci.yml

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm install
      - run: npm test
```

Key	What it means
`on`	Which event(s) trigger this workflow — push, pull_request, schedule, workflow_dispatch…
`jobs`	One or more named jobs; each runs independently (in parallel by default).
`runs-on`	The runner's OS image — ubuntu-latest, windows-latest, macos-latest.
`steps`	An ordered list of actions (reusable, e.g. actions/checkout) or shell commands (run:).
`secrets`	Encrypted values (API keys, tokens) injected via ${{ secrets.NAME }}, never printed in logs.

Common triggers

- `push` — runs on every push to matching branches.
- `pull_request` — runs on every PR open/update, great for required checks.
- `schedule` (cron) — runs on a timer, e.g. nightly builds.
- `workflow_dispatch` — a manual "Run workflow" button in the Actions tab.

NOTE GitHub-hosted runners include generous free minutes for public repos (and a monthly allowance for private ones). Check current limits under Settings › Billing › Plans and usage.

10 Projects

Kanban-style boards and planning views

A **Project** is a flexible planning board that pulls in Issues and PRs from one or more repos. Think Trello, built into GitHub — drag cards across columns as work moves along.

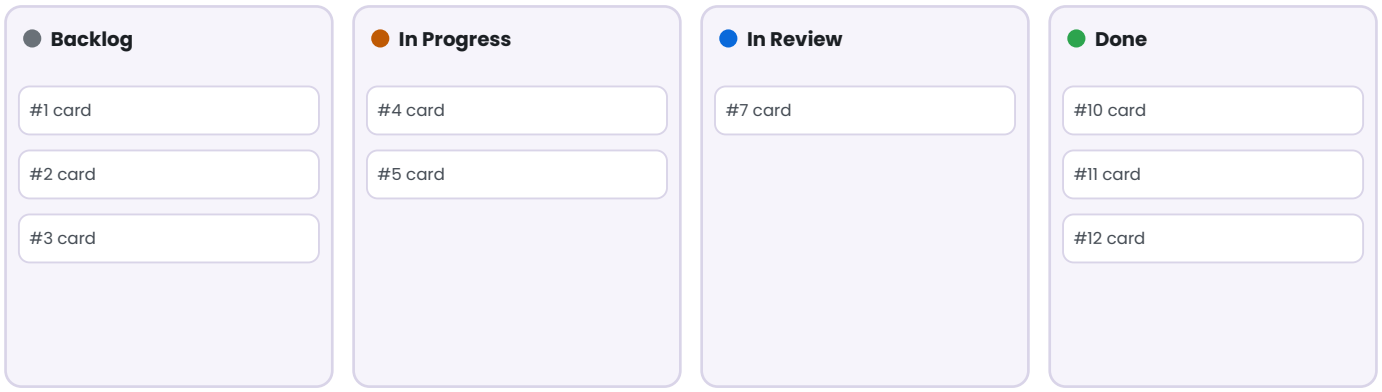


Fig. A board view — columns are statuses; cards are linked Issues or PRs.

Views: more than just a board

View	What it shows
`Board`	Kanban columns by status (Todo / In Progress / Done, or any custom field).
`Table`	A spreadsheet-like grid — sort, filter, and group by any field.
`Roadmap`	A timeline/Gantt-style view using start and target-date fields.

Building a Project

1. **Create a Project**
   From your profile/org, or a repo's Projects tab › New project.
2. **Add custom fields**
   Status, Priority, Size, Iteration, or any text/number/date field you define.
3. **Add items**
   Link existing Issues/PRs, or add draft items (plain cards with no repo yet).
4. **Automate**
   Built-in workflows: auto-move to "Done" when a linked PR merges, auto-archive closed items.

TIP Projects (the modern version, sometimes called "Projects v2") can span **multiple repositories** — perfect for tracking a feature that touches a frontend repo and a backend repo at once.

11 Wiki

Long-form documentation, right next to the code

Every repo can have its own **Wiki** — a set of Markdown pages for documentation that's too long-form for a README: architecture notes, onboarding guides, FAQs, meeting notes.

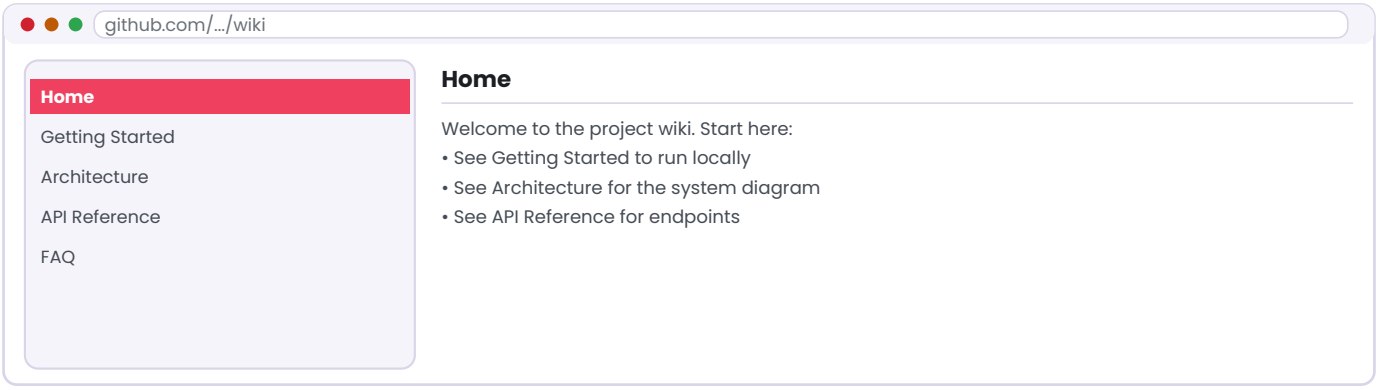


Fig. The Wiki is its own mini-site: a page sidebar plus rendered Markdown content.

README vs. Wiki vs. Pages — which goes where

Where	What belongs there
`README.md`	The front door. Quick start, what the project does, badges, links elsewhere.
`Wiki`	Deeper docs that change less often: architecture, design decisions, FAQs.
`GitHub Pages`	A published website — for user-facing docs sites, demos, or marketing (Section 13).

Working with the Wiki

- Edit pages right in the browser, or clone the wiki itself as its own Git repo:

```
terminal — every repo's wiki is secretly its own .wiki.git repo
$ git clone https://github.com/you/repo.wiki.git
```

- Wikis can be public-editable or restricted to collaborators, under the repo's Settings.

NOTE

A Wiki is not searched by default the same way code is — for docs you want indexed by search engines and linked publicly, GitHub Pages is usually the better fit.

12 Releases & Tags

Shipping versions people can download

A **Release** packages a specific point in your history (a tag) with release notes and, optionally, downloadable files (binaries, zipped source, installers) — the "official" way to ship a version.



Fig. Tags pin a commit forever; Releases wrap a tag with notes and downloadable assets.

Semantic Versioning (semver)

Most projects version as `MAJOR.MINOR.PATCH`, e.g. `v2.1.3`:

Part	Means
`MAJOR`	Breaking changes — code using this library may need updates.
`MINOR`	New features, backward-compatible.
`PATCH`	Bug fixes only, backward-compatible.

Creating a release

1. **Tag a commit**

   git tag -a v1.2.0 -m "Release 1.2.0" then git push origin v1.2.0 — or let GitHub create the tag for you in the next step.

2. **Go to Releases › Draft a new release**

   Pick the tag (existing or new), target branch, and a title.

3. **Write release notes**

   Click "Generate release notes" to auto-build a changelog from merged PRs.

4. **Attach assets (optional)**

   Upload built binaries, installers, or a changelog file.

5 Publish

Mark as a pre-release if it's a beta; otherwise it becomes “Latest.”

terminal

```
$ git tag -a v1.2.0 -m "Release 1.2.0"
$ git push origin v1.2.0
$ gh release create v1.2.0 --generate-notes
```

CAUTION Deleting a tag locally does **not** delete it on GitHub, and vice versa — `git push origin --delete v1.2.0` removes the remote tag explicitly.

13 GitHub Pages

Free static website hosting from a repo

GitHub Pages publishes static files (HTML/CSS/JS, or a static-site generator's output) straight from a repo to a live URL — free hosting for docs sites, portfolios, project pages, and demos.



Fig. Pages serves static files straight from a branch or folder in your repo.

Three ways to publish

Method	How it works
`Deploy from a branch`	Pick a branch (often gh-pages) and folder (/ or /docs). Simplest, good for plain HTML.
`GitHub Actions`	A workflow builds your site (Jekyll, Hugo, React, etc.) and deploys the output automatically on every push.
`Jekyll (built-in)`	GitHub can run Jekyll for you automatically if it detects a _config.yml — zero extra setup.

Turning it on

1. **Go to Settings › Pages**
   In the repo you want to publish from.
2. **Pick a source**
   Branch + folder, or “GitHub Actions” for a custom build.
3. **Save**
   GitHub builds and gives you a URL: https://username.github.io/repo-name/
4. **(Optional) custom domain**
   Add a CNAME file, or set it under Settings › Pages › Custom domain.

.github/workflows/pages.yml — deploy a built site via Actions

```yaml
name: Deploy Pages
on: { push: { branches: [main] } }
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with: { path: ./dist }
      - uses: actions/deploy-pages@v4
```

NOTE

A user/organization site (`username.github.io`) must live in a repo named exactly that; a project site can live in any repo and publishes under `username.github.io/repo-name`.

14 Packages

Publish & install packages via GitHub

GitHub Packages is a package registry built into GitHub — host npm, Docker/container images, Maven, NuGet, RubyGems and more, right next to the code that builds them, with the same permissions as your repo.

Ecosystem	Notes
`npm`	JavaScript/Node packages — published to npm.pkg.github.com.
`Docker / Container`	OCI images, pushed to the GitHub Container Registry (ghcr.io).
`Maven / Gradle`	Java packages.
`NuGet`	.NET packages.
`RubyGems`	Ruby gems.

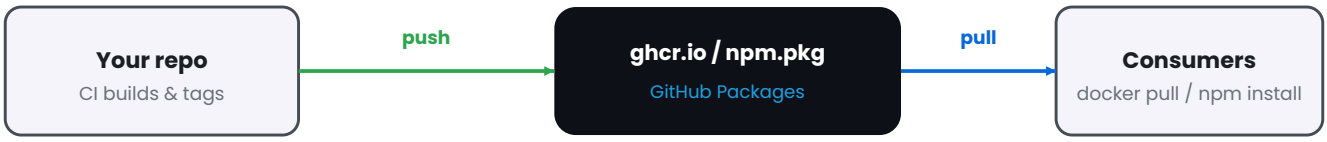


Fig. CI builds and pushes a package; anyone with access can pull it.

Publishing a Docker image from Actions

workflow step

```
- name: Log in to GHCR
  run: echo ${{ secrets.GITHUB_TOKEN }} | docker login ghcr.io -u $ --password-stdin
- name: Build and push
  run: |
    docker build -t ghcr.io/you/app:latest .
    docker push ghcr.io/you/app:latest
```

Installing an npm package from GitHub Packages

terminal

```
# .npmrc
@you:registry=https://npm.pkg.github.com

$ npm install @you/some-package
```

TIP

Packages inherit the visibility and permissions of the repo they're published from by default — a private repo's packages are private too, unless you change that explicitly.

15 Codespaces

A full dev environment, in your browser

A **Codespace** is a complete, cloud-hosted dev environment for a repo — a VS Code interface, a terminal, and all your dependencies pre-installed — running in the browser (or locally in VS Code). No local setup required.

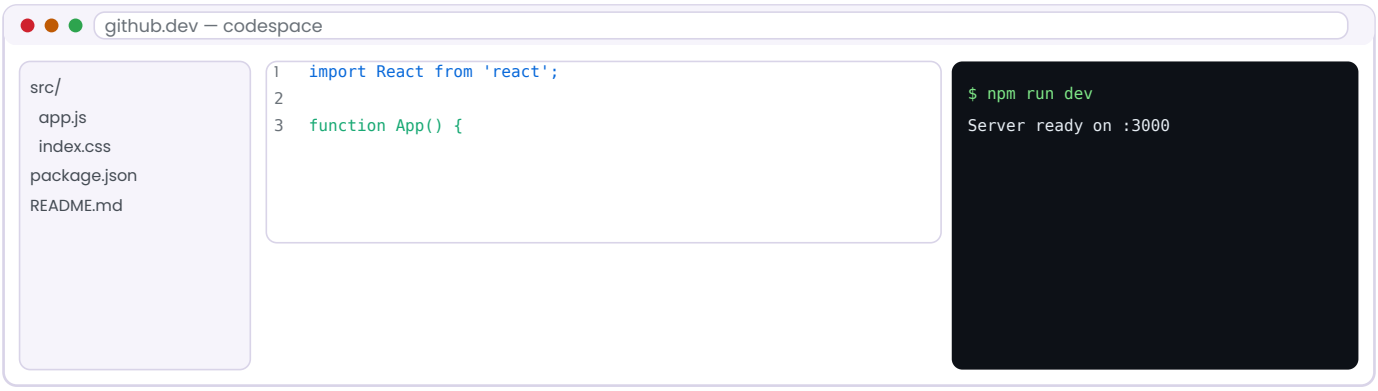


Fig. File tree, editor, and terminal — a full IDE running in the cloud.

Starting a Codespace

- From any repo: green **Code** button › Codespaces tab › **Create codespace on main**.
- Or press `.` for the lighter web editor, then "Continue Working on" › Codespace for the full VM.
- Or from the CLI: `gh codespace create`.

devcontainer.json — defining the environment

.devcontainer/devcontainer.json

```
{
  "image": "mcr.microsoft.com/devcontainers/javascript-node:20",
  "forwardPorts": [3000],
  "postCreateCommand": "npm install"
}
```

NOTE Codespaces include free monthly hours on the Free plan, billed by machine size beyond that — stop (don't just close the tab) an idle Codespace to avoid burning hours unnecessarily.

16 Copilot Overview

AI pair-programmer across GitHub

GitHub Copilot is an AI assistant woven through the GitHub ecosystem — in your editor, on the website, and inside pull requests. It's a tool you review and steer, not an autopilot.

1 Completions

Inline code suggestions as you type, in your editor.

2 Chat

Ask questions about your code, get explanations, generate tests.

3 PR Summaries

Auto-drafts a description of what a PR changes and why.

Fig. Copilot shows up in three main places: your editor, chat, and pull requests.

Surface	What it does
`Copilot in your editor`	Ghost-text suggestions as you type; accept with Tab. Works in VS Code, JetBrains, Neovim, Codespaces.
`Copilot Chat`	A conversational panel — explain this function, write tests, fix this error, refactor this.
`Copilot in github.com`	Chat available directly on the website — ask about a repo, a file, an issue.
`Copilot PR summaries`	Auto-generates a "Walkthrough" describing the diff when opening a PR.
`Copilot code review`	Can leave automated review comments on a PR, like an extra reviewer.

Getting started

1. **Check your plan**

   Individual (paid, free tier available), Business, or Enterprise — org admins can grant seats under org Settings › Copilot.

2. **Install the extension**

   In VS Code / JetBrains: search "GitHub Copilot" in the extensions panel, sign in with GitHub.

3 **Start typing, or open Chat**

Suggestions appear automatically; Ctrl/Cmd+I or the Chat panel for conversation.

CAUTION Treat Copilot's output the way you'd treat a junior contributor's PR: read it, test it, and understand it before committing — it can be confidently wrong.

17 Security Essentials

Protect branches, dependencies, and secrets

GitHub ships several free, built-in security features. Turning them on is mostly a matter of knowing they exist — this section covers the three every repo should have.

1 Branch Protection

Block direct pushes; require review + passing checks before merge.

2 Dependabot

Auto-flags (and can auto-PR fixes for) vulnerable dependencies.

3 Secret Scanning

Detects leaked API keys/tokens in commits and blocks new ones on push.

Fig. Three layers of protection, each free on public repos (and available on private with a plan).

Branch protection rules

Settings › Branches › Add rule, targeting `main` (or a pattern like `release/*`):

Rule	What it enforces
`Require a pull request before merging`	No one (including admins, optionally) can push straight to main.
`Require approvals`	At least N reviewers must approve before the Merge button unlocks.
`Require status checks to pass`	CI (Actions) must succeed — pick which checks are required.
`Require branches to be up to date`	Your branch must include the latest main before merging.
`Require signed commits`	Only cryptographically signed commits are accepted.
`Include administrators`	Apply all the above to repo admins too, not just everyone else.



Dependabot

- **Dependabot alerts** — scans your dependency manifests (package.json, requirements.txt, go.mod…) against known vulnerability databases, flags affected packages.
- **Dependabot security updates** — automatically opens a PR bumping a vulnerable dependency to a patched version.
- **Dependabot version updates** — opt-in, keeps dependencies current even without a known CVE, configured via a file:

.github/dependabot.yml

```
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule: { interval: "weekly" }
```

Secret scanning

GitHub scans every push for recognizable secret formats (AWS keys, Stripe keys, database connection strings, and hundreds more). On public repos this is always on; **push protection** can additionally block the push itself before a secret ever lands in history.

WATCH OUT

If a secret is ever committed — even briefly — treat it as compromised: rotate/revoke it immediately. Removing it from a later commit does not erase it from Git history.

18

GitHub CLI (gh)

Everything on GitHub, without leaving the terminal

gh is GitHub's official command-line tool. It wraps the website's features — repos, issues, PRs, Actions runs, releases — as fast, scriptable commands. Install from `cli.github.com`, then authenticate once:

terminal — install & authenticate

```
$ brew install gh          # macOS
$ winget install GitHub.cli   # Windows
$ sudo apt install gh      # Linux (after adding GitHub's apt repo)

$ gh auth login            # one-time browser-based login
$ gh auth status           # confirm you're logged in
```

repo — repositories

Command	What it does
`gh repo create name`	Create a new repo (prompts for public/private, add README…).
`gh repo clone owner/repo`	Clone a repo — shorthand for git clone with the right URL.
`gh repo fork owner/repo`	Fork a repo, optionally --clone to pull it down immediately.
`gh repo view --web`	Open the current repo's GitHub page in your browser.
`gh repo list owner`	List repos under a user or org.
`gh repo delete owner/repo`	Delete a repo (asks for confirmation).

issue — issues

Command	What it does
`gh issue list`	List issues in the current repo (--assignee, --label to filter).
`gh issue create`	Open a new issue interactively, or --title/--body for non-interactive.
`gh issue view 42`	Show issue #42; add --web to open it in the browser instead.
`gh issue close 42`	Close an issue; gh issue reopen 42 to reopen it.
`gh issue comment 42`	Add a comment, e.g. --body "fixed in v1.2".

pr — pull requests

Command	What it does
`gh pr create --fill`	Open a PR using the branch's commits for the title/body automatically.
`gh pr list`	List open PRs (--state all/closed/merged to change scope).
`gh pr view 42 --web`	Open PR #42 in the browser.
`gh pr checkout 42`	Check out a PR's branch locally — great for testing someone else's change.
`gh pr diff 42`	See the diff for a PR right in the terminal.
`gh pr review 42 --approve`	Approve (or --request-changes / --comment) a PR.
`gh pr merge 42 --squash`	Merge a PR — --merge, --squash, or --rebase for the method.
`gh pr status`	Show PRs relevant to you: yours, requesting your review, assigned to you.

workflow & run — GitHub Actions

Command	What it does
`gh workflow list`	List workflows defined in the repo.
`gh workflow run ci.yml`	Manually trigger a workflow_dispatch workflow.
`gh run list`	List recent workflow runs and their status.
`gh run view 123 --log`	View the full log output of a specific run.
`gh run watch`	Live-follow the most recent in-progress run.

release, gist & codespace

Command	What it does
`gh release create v1.2.0 --generate-notes`	Create a release from an existing or new tag.
`gh release list`	List all releases in the repo.
`gh gist create file.py`	Publish a file as a Gist (--public for a public one).
`gh codespace create`	Create a new Codespace for the current repo.
`gh codespace ssh`	SSH into a running Codespace from your terminal.

api & extensions — going further

`gh api` gives you raw access to GitHub's REST/GraphQL API using your existing authentication — useful for anything without a dedicated command yet:

terminal

```
$ gh api repos/octo-org/app/issues --paginate
$ gh api graphql -f query='{ viewer { login } }'

$ gh extension install owner/gh-extension-name
$ gh extension list
```

gh vs. git — who does what

Tool	Role
`git`	Version control itself: commit, branch, merge, log, diff — works with any Git host.
`gh`	GitHub-specific actions: creating PRs/issues, viewing Actions runs, managing releases.

A realistic daily flow, start to finish

terminal — git + gh working together

```
$ gh repo clone sharadverma638/app && cd app
$ git switch -c feature/dark-mode
$ # ...make changes...
$ git add . && git commit -m "Add dark mode toggle"
$ git push -u origin feature/dark-mode
$ gh pr create --fill
$ gh pr view --web          # share the link / sanity check
$ gh pr checks               # wait for CI
$ gh pr merge --squash --delete-branch
```

TIP Almost every `gh` command accepts `--web` to open the equivalent page in your browser instead — handy when you just want to double-check something visually.

19

GitHub Desktop Walkthrough

All the core workflow, with no terminal

GitHub Desktop is a free visual app for the everyday Git/GitHub workflow — clone, commit, branch, push, pull, and open PRs, all by clicking instead of typing. Great for anyone who wants the benefits of version control without memorizing commands yet.



Fig. The Desktop window: changed files on the left, a diff preview on the right.

The core workflow

1. **Install & sign in**
   desktop.github.com — install, then sign in with your GitHub account.
2. **Clone a repository**
   File › Clone repository — pick from your GitHub repos or paste a URL.
3. **Make changes in your editor**
   Desktop watches the folder and lists every changed file automatically.
4. **Stage, write a summary, commit**
   Check the files you want, describe the change, click Commit to main.
5. **Push to GitHub**
   Click “Push origin” — uploads your commits to the remote.
6. **Branch & open a PR**
   Current Branch › New Branch; after pushing, “Create Pull Request” opens your browser pre-filled.

Feature	What it's for
`Current Branch`	Switch or create branches — the dropdown at the top-center.
`Fetch / Pull / Push`	One button, label changes based on state (Fetch › Pull › Push).
`History tab`	See past commits, right next to the Changes tab.
`Repository › Open in Terminal`	Drop to the command line for anything Desktop doesn't cover.

NOTE

GitHub Desktop and the terminal share the same underlying Git repo — you can freely switch between clicking in Desktop and typing `git` commands on the same project.

20 Best Practices

Habits that keep repos and teams healthy

None of these are enforced by GitHub — they're habits that separate a repo that's pleasant to work in from one that isn't.

Repo hygiene

- **Write a real README** — what it does, how to run it, how to contribute. The first thing anyone sees.
- **Use a .gitignore from day one** — never commit build output, dependencies, or local config.
- **Pick a license** early if the repo is public — "no license" legally means no one may reuse your code.
- **Keep main always deployable** — merge only finished, reviewed, passing work.

Issues & PRs

- **One PR, one concern** — small, focused PRs get reviewed faster and are easier to revert.
- **Link issues from PRs** — "Fixes #42" keeps history traceable and auto-closes the issue.
- **Write descriptive titles** — "Fix navbar overlap on mobile Safari" beats "fix bug."
- **Review your own diff first** — catch debug prints and leftover comments before a human has to.

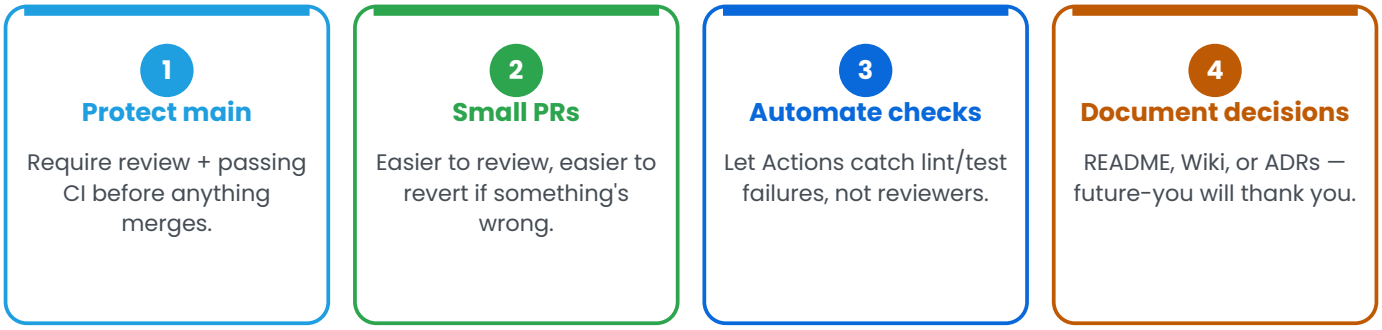


Fig. Four habits that compound over a project's lifetime.

Security & secrets

- Never commit `.env` files or API keys — use repo/org **Secrets** for CI, and a local .env (gitignored) for development.
- Turn on Dependabot alerts and branch protection on every repo that matters (Section 17).
- Use SSH keys or fine-scoped Personal Access Tokens, never your account password, for Git operations.

Organizing at scale

Tool	Why it helps
`CODEOWNERS file`	Auto-requests the right reviewers based on which files a PR touches.
`Issue/PR templates`	.github/ISSUE_TEMPLATE and PULL_REQUEST_TEMPLATE.md keep submissions consistent.
`CONTRIBUTING.md`	Sets expectations for outside contributors before they open a PR.
`Labels & milestones`	Keep a consistent, small label set — too many labels is as bad as none.

Using GitHub in the AI era

AI tools (Copilot and others) write code fast — but PRs, reviews, and branch protection matter **more**, not less, when code arrives quickly. Never merge AI-assisted changes without reading the diff, and keep CI as a non-negotiable gate regardless of who (or what) wrote the code.

KEY IDEA

A repo with clean branch protection, small PRs, and a real README scales to a team of 50 the same way it works for a team of one — the habits are the same, just enforced earlier.

21 Cheat Sheet & Glossary

Every command, one place

Quick copy-paste reference for everything in this guide.

gh auth & setup

`gh auth login`	One-time browser login
`gh auth status`	Confirm logged-in account
`gh repo clone o/r`	Clone a repo
`gh repo create name`	Create a new repo
`gh repo fork o/r`	Fork a repo

gh issue

`gh issue list`	List issues
`gh issue create`	Open a new issue
`gh issue view 42`	Show issue #42
`gh issue close 42`	Close an issue

gh pr

`gh pr create --fill`	Open a PR from branch commits
`gh pr checkout 42`	Check out a PR locally
`gh pr review --approve`	Approve a PR
`gh pr merge --squash`	Merge (squash) a PR
`gh pr status`	PRs relevant to you

gh run / workflow

`gh workflow run ci.yml`	Trigger a workflow
`gh run list`	Recent Actions runs
`gh run watch`	Follow a run live

gh release / codespace

`gh release create v1.0 --generate-notes`	Create a release
`gh codespace create`	New Codespace
`gh codespace ssh`	SSH into a Codespace

Repository basics

`Fork`	top-right of any repo › your own copy
`Star`	bookmark a repo you like
`Watch`	get notified of activity
`New repo`	github.com/new

Branch protection (web)

`Settings › Branches`	add a protection rule
`Require PR + review`	block direct pushes
`Require status checks`	CI must pass to merge

Merge methods

`Merge commit`	keeps every commit + a merge commit
`Squash and merge`	one clean commit (most common)
`Rebase and merge`	linear history, no merge commit

Secrets keyword to close issues

`Fixes #42 / Closes #42`	in a PR description, auto-closes on merge

GitHub Pages

`Settings › Pages`	pick branch/folder or Actions source
`user.github.io`	user/org site, repo must match name
`user.github.io/repo`	project site, any repo name

Common gh flags

`--web`	open the equivalent page in browser
`--fill`	use branch commits as title/body
`--paginate`	fetch every page from gh api

Mini glossary

Repository (repo)	A project's folder plus its entire Git history, hosted on GitHub.
Fork	Your own personal copy of someone else's repository, on your account.
Clone	A local copy of a repo downloaded to your machine.
Issue	A trackable item on GitHub — a bug, task, or request for discussion.
Pull Request (PR)	A request to merge changes from one branch into another, with review built in.
Branch protection	Rules that block direct pushes and require review/checks before merging to a branch.
Merge commit	A merge method that keeps every commit and adds a commit joining the branches.
Squash and merge	A merge method that collapses a PR's commits into a single commit on the base branch.
Review	A reviewer's verdict on a PR: Comment, Approve, or Request changes.
Draft PR	A pull request marked not-yet-ready for full review.
Label	A colored tag on an Issue/PR for categorization (bug, enhancement…).
Milestone	A grouping of Issues/PRs toward a shared deadline or release.
Discussion	An open-ended conversation space separate from Issues.
GitHub Actions	GitHub's built-in automation/CI-CD system, driven by YAML workflow files.
Workflow	A YAML file under .github/workflows/ defining jobs that run on triggers.
Runner	The machine (hosted or self-hosted) that executes a workflow's jobs.
Project	A flexible Kanban/table/roadmap board linking Issues and PRs to a plan.
Release	A packaged, versioned snapshot of the repo with notes and optional downloadable assets.
Tag	A permanent pointer to one specific commit, usually marking a release.
Semantic Versioning	A MAJOR.MINOR.PATCH versioning convention signaling the size of a change.
GitHub Pages	Free static website hosting published directly from a repo.
GitHub Packages	A package registry (npm, Docker, Maven…) built into GitHub, scoped to a repo/org.
Codespace	A full cloud-hosted dev environment for a repo, accessible from the browser.
devcontainer.json	A config file defining a Codespace's image, ports, and setup commands.
Copilot	GitHub's AI pair-programmer: editor completions, chat, and PR summaries.
Dependabot	Automated dependency vulnerability alerts and update PRs.
Secret scanning	GitHub's detection (and optional blocking) of leaked credentials in pushes.
Personal Access Token (PAT)	A scoped credential used instead of a password for Git/API authentication.
SSH key	A cryptographic key pair used to authenticate Git operations without a password.
gh	GitHub's official command-line tool for repos, issues, PRs, Actions, and more.
Upstream	In a fork workflow, the original repository you forked from.
Origin	The default name for the remote a repo was cloned from (often your fork).
CODEOWNERS	A file that auto-assigns reviewers based on which files a PR touches.

TRY IT

Practice loop: create a repo, open an Issue, branch, commit, push, and open a PR against your own repo — then merge it. Once that full loop feels automatic, add Actions, then explore the gh CLI for the same flow without the browser.

That's GitHub, start to finish. The companion Git Notes guide covers the underlying tool itself in depth — branching, rebase, stash, and the full command reference.